AWS

# VPC Peering

## Connect two VPCs. Prove private connectivity.

Student project report: EC2, custom route tables, security groups and a bastion host




**AWS ARCHITECT ASSOCIATE | PROJECT 01 | EWB CLOUD LABS**

| | |
|---|---|
| Name | Abhishek |
| Batch | _____ |
| Project title | AWS VPC Peering |
| Region | Asia Pacific (Mumbai) / ap-south-1 |
| Completion date | 28 October 2026 |

LEARN. ELEVATE. EMPOWER.

# 1. Project overview

EWB has an application in VPC A and a second workload in VPC B. Both application servers must stay private. I built two VPCs in one account and one Region, connected them with a VPC peering connection, and proved private connectivity between App-A (10.10.2.10) and App-B (10.20.1.10) using their private IPv4 addresses. One public bastion host in VPC A is the only entry point for administration. I built the lab with a reproducible AWS CLI script (`ewb-vpc-peering.sh`) and verified every resource in the AWS console.

## What this report demonstrates

| Goal | Where in this report |
|---|---|
| Plan non-overlapping VPC and subnet CIDRs | Section 2 and 3 |
| Public and private subnets with explicit route table associations | Section 4 |
| Peering, return routes and least-privilege inbound rules | Sections 5 and 6 |
| Failed test before peering, successful tests after, and a controlled failure | Sections 8 and 9 |
| Forward path, return path and viva answers | Sections 10 and 11 |
| Cleanup evidence and lessons learned | Sections 12 and 13 |

# 2. Architecture

One internet entry point (the internet gateway on VPC A and a public bastion). Two private workloads. A direct peering connection between the two VPCs. VPC B has no internet gateway. Peering does not give VPC B internet access through VPC A.

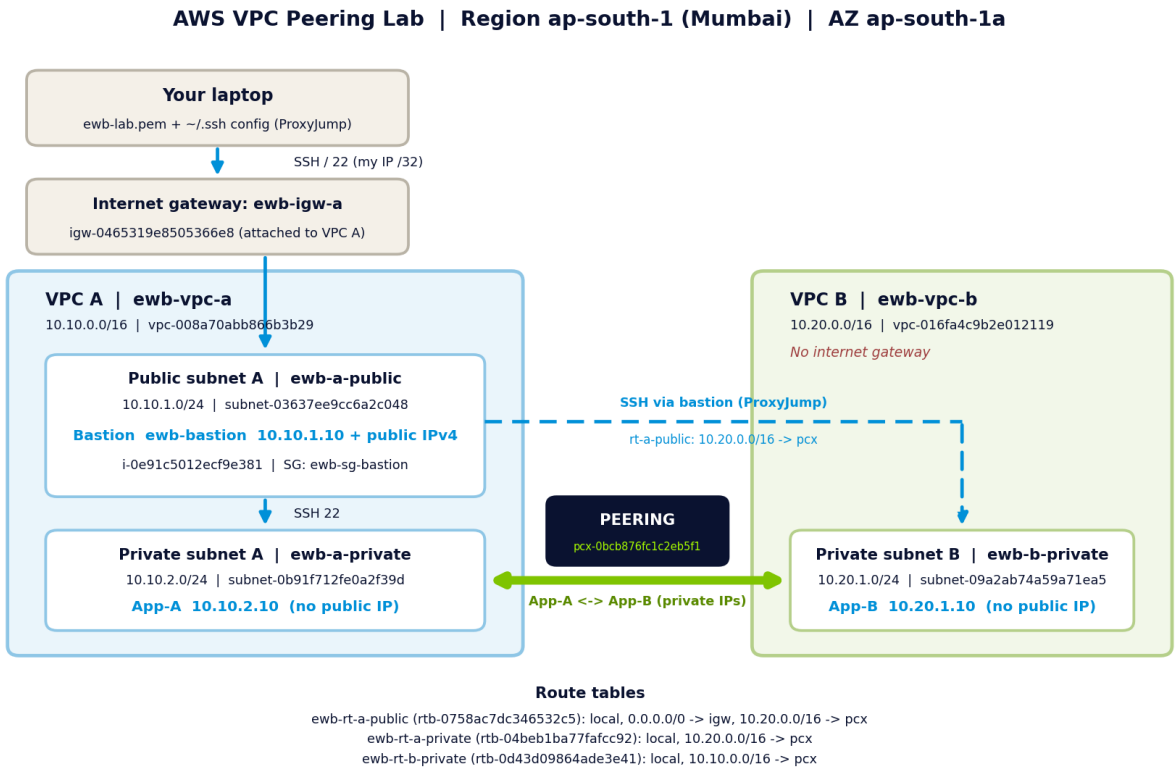



**Figure 1.** Architecture diagram with my actual resource IDs and private IPs.

**Administration path:** laptop > bastion > App-A or App-B using SSH ProxyJump. **Project traffic:** App-A and App-B over private IPs and the peering connection.

# 3. Resource inventory

Actual IDs from my lab account (Region ap-south-1, Availability Zone ap-south-1a). All resources are tagged `Project = EWB-VPC-Peering` and `Student = abhishek`. The account ID and bastion public IP are redacted.

| Resource (Name tag) | ID | CIDR / private IP |
|---|---|---|
| VPC A (ewb-vpc-a) | `vpc-008a70abb866b3b29` | 10.10.0.0/16 |
| VPC B (ewb-vpc-b) | `vpc-016fa4c9b2e012119` | 10.20.0.0/16 |
| Public subnet (ewb-a-public) | `subnet-03637ee9cc6a2c048` | 10.10.1.0/24 |
| Private subnet A (ewb-a-private) | `subnet-0b91f712fe0a2f39d` | 10.10.2.0/24 |
| Private subnet B (ewb-b-private) | `subnet-09a2ab74a59a71ea5` | 10.20.1.0/24 |
| Internet gateway (ewb-igw-a) | `igw-0465319e8505366e8` | Attached to VPC A |
| Route table (ewb-rt-a-public) | `rtb-0758ac7dc346532c5` | Assoc: ewb-a-public |
| Route table (ewb-rt-a-private) | `rtb-04beb1ba77fafcc92` | Assoc: ewb-a-private |
| Route table (ewb-rt-b-private) | `rtb-0d43d09864ade3e41` | Assoc: ewb-b-private |
| Security group (ewb-sg-bastion) | `sg-09d341edc610eff62` | VPC A |
| Security group (ewb-sg-app-a) | `sg-03a9cb34a7c230ead` | VPC A |
| Security group (ewb-sg-app-b) | `sg-04431a53275beffb0` | VPC B |
| Peering connection (ewb-a-b-peer) | `pcx-0bcb876fc1c2eb5f1` | Requester VPC A, accepter VPC B |
| Instance (ewb-bastion) | `i-0e91c5012ecf9e381` | 10.10.1.10 + public IPv4 (redacted) |
| Instance (ewb-app-a) | `i-00187fcc472c1dd81` | 10.10.2.10 / no public IP |
| Instance (ewb-app-b) | `i-0dd5bd012bab26179` | 10.20.1.10 / no public IP |
| Key pair | `ewb-lab (RSA)` | Private key stays on my laptop |

## CIDR plan

10.10.0.0/16 (VPC A) and 10.20.0.0/16 (VPC B) do not overlap, so peering can be created. Each /24 subnet sits inside its own VPC range. Different subnet ranges alone would not fix overlapping VPC CIDRs.

# 4. Subnets, internet gateway and route tables

I created VPC A and VPC B with the "VPC only" option so that I control every subnet and route. The public subnet has a default route to the internet gateway. Both private subnets have no internet route and keep public IPv4 auto-assignment disabled.

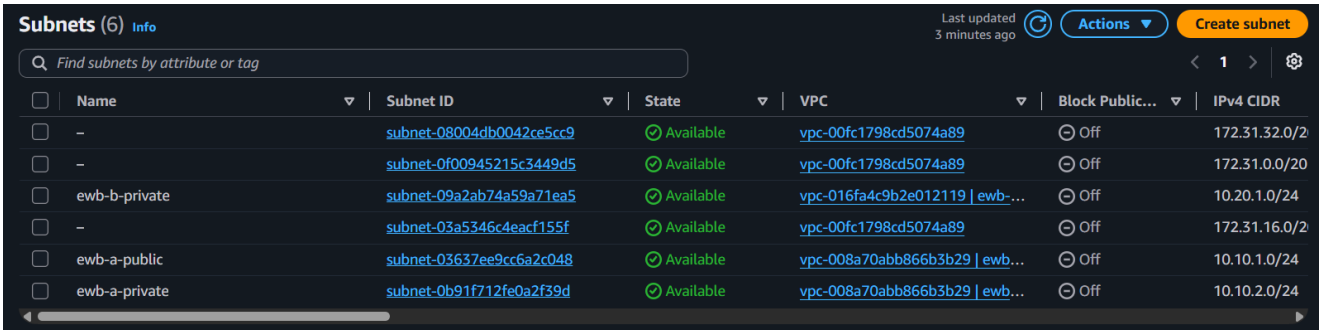



**Figure 2.** Console / VPC > Subnets. ewb-a-public 10.10.1.0/24, ewb-a-private 10.10.2.0/24, ewb-b-private 10.20.1.0/24. Result: three subnets Available, Block Public Access Off.

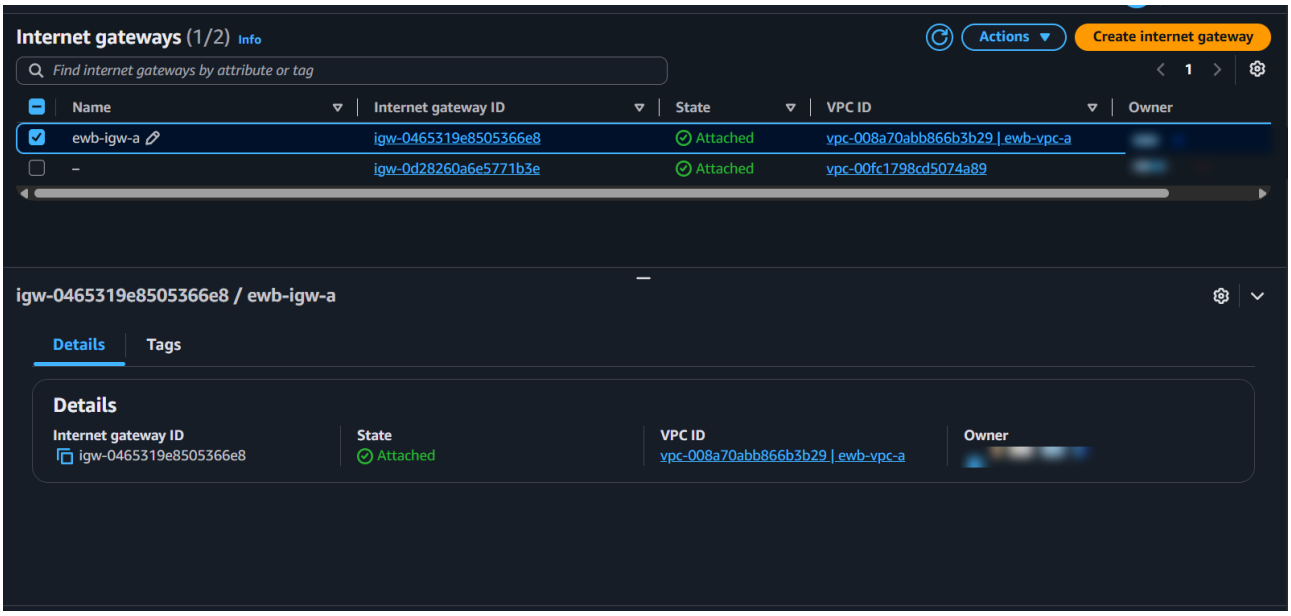



**Figure 3.** Console / VPC > Internet gateways. ewb-igw-a is Attached to ewb-vpc-a. Account ID blurred.

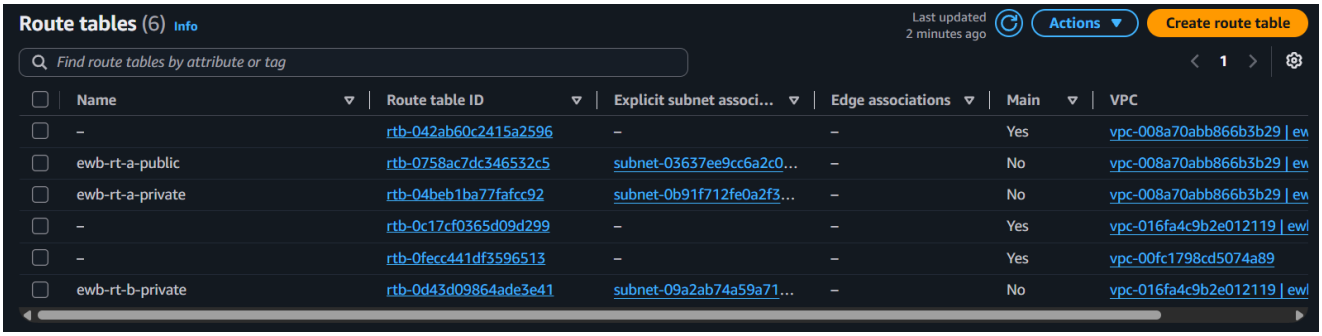



**Figure 4.** Console / VPC > Route tables. Each custom route table has one explicit subnet association.

## Route tables and subnet associations

| Route table | Associated subnet | Destination | Target | Purpose |
|---|---|---|---|---|
| ewb-rt-a-public | ewb-a-public | 10.10.0.0/16<br>0.0.0.0/0<br>10.20.0.0/16 | local<br>igw-0465319e8505366e8<br>pcx-0bcb876fc1c2eb5f1 | Bastion to App-A<br>Bastion internet access<br>Bastion SSH to App-B |
| ewb-rt-a-private | ewb-a-private | 10.10.0.0/16<br>10.20.0.0/16 | local<br>pcx-0bcb876fc1c2eb5f1 | Existing VPC A route<br>App-A to VPC B |
| ewb-rt-b-private | ewb-b-private | 10.20.0.0/16<br>10.10.0.0/16 | local<br>pcx-0bcb876fc1c2eb5f1 | Existing VPC B route<br>Return path to VPC A |

The public subnet also needs a peering route. ProxyJump makes the bastion open its own SSH connection to App-B, so the bastion subnet must know how to reach VPC B. The more specific 10.20.0.0/16 route is selected instead of the internet default route.

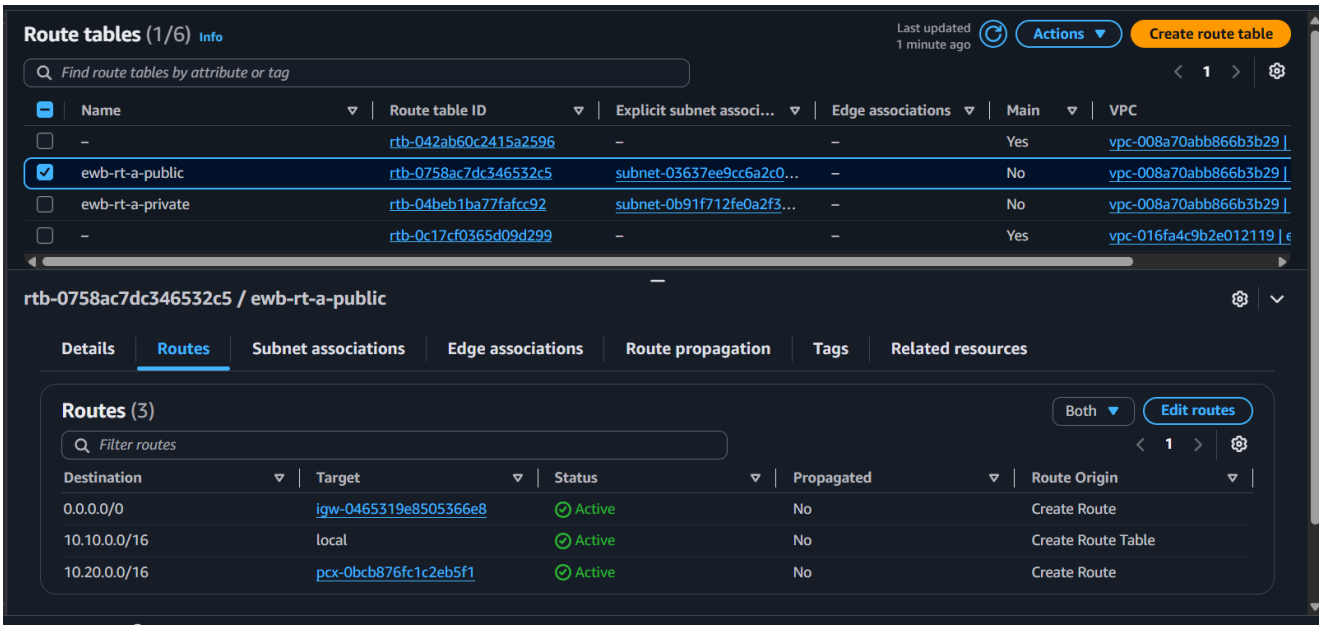



**Figure 5.** Console / ewb-rt-a-public > Routes tab. Result: 0.0.0.0/0 -> igw, 10.10.0.0/16 local, 10.20.0.0/16 -> pcx, all Active.

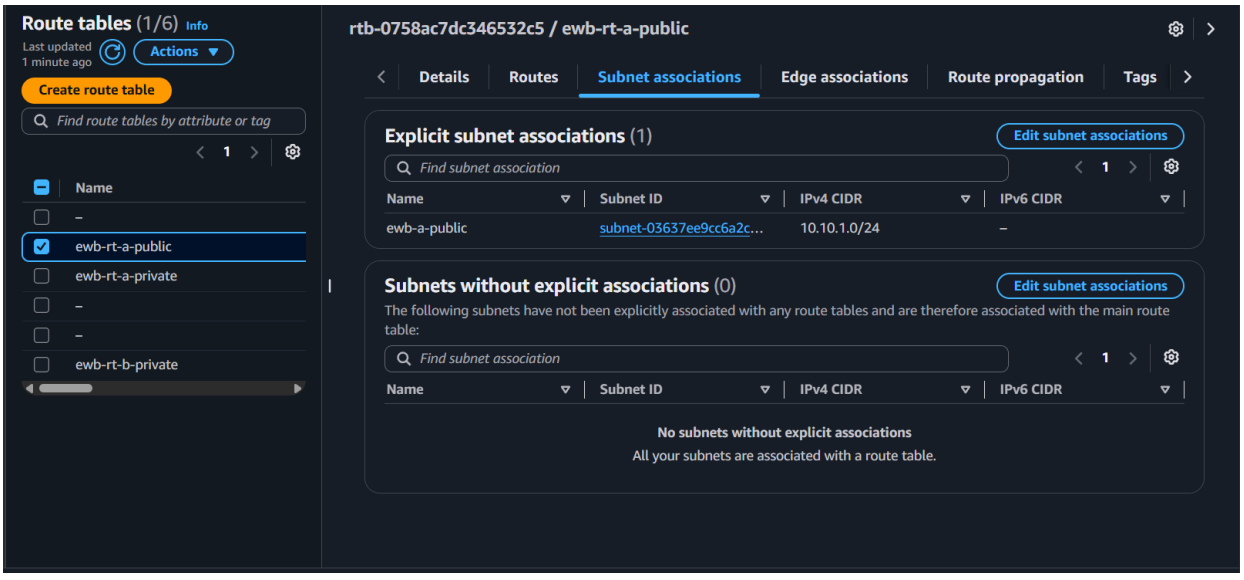



**Figure 6.** Console / ewb-rt-a-public > Subnet associations. Result: ewb-a-public 10.10.1.0/24 explicitly associated; no subnets left on the main table.

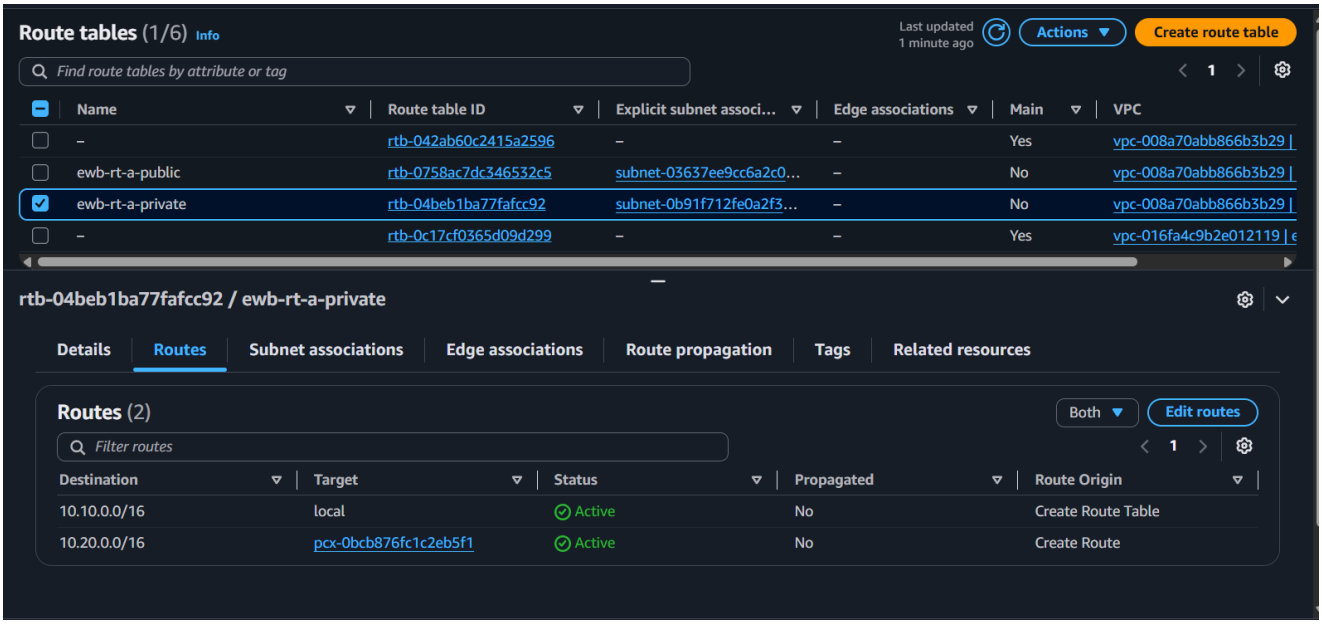



**Figure 7.** Console / ewb-rt-a-private > Routes tab. Result: local plus 10.20.0.0/16 -> pcx. No internet default route.

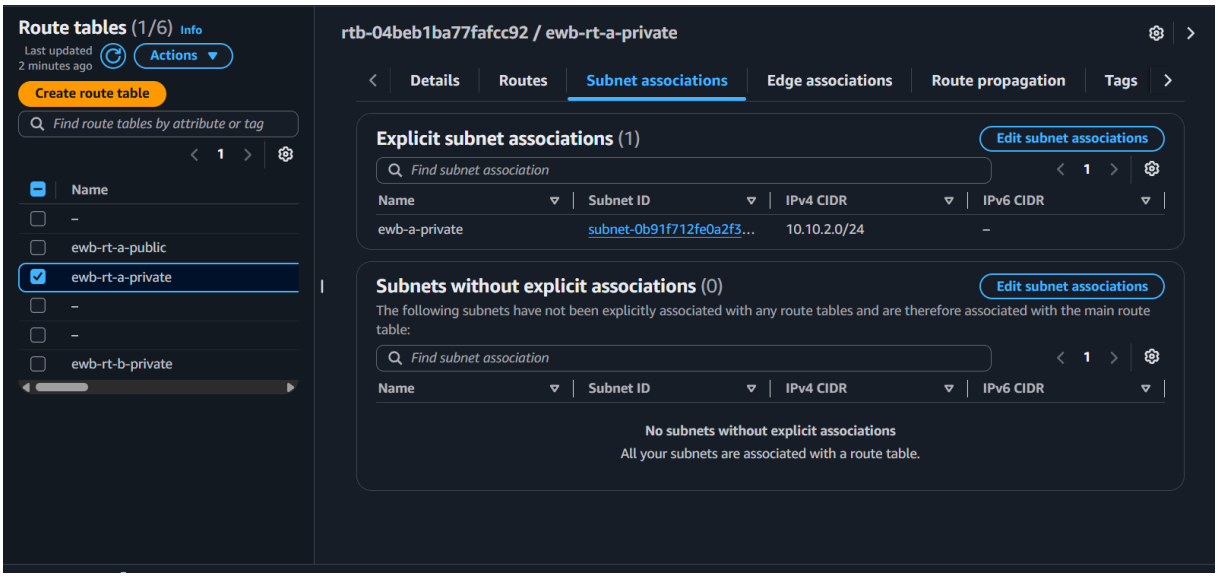



**Figure 8.** Console / ewb-rt-a-private > Subnet associations. Result: ewb-a-private 10.10.2.0/24 explicitly associated.

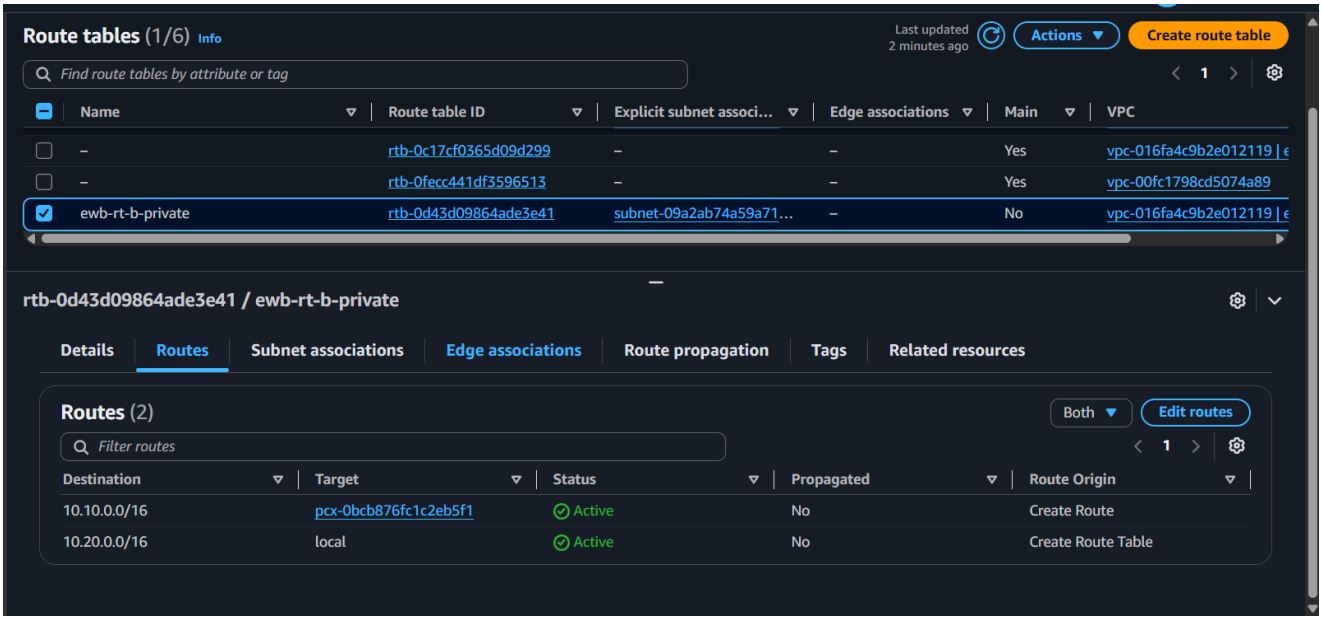



**Figure 9.** Console / ewb-rt-b-private > Routes tab. Result: local plus return route 10.10.0.0/16 -> pcx.

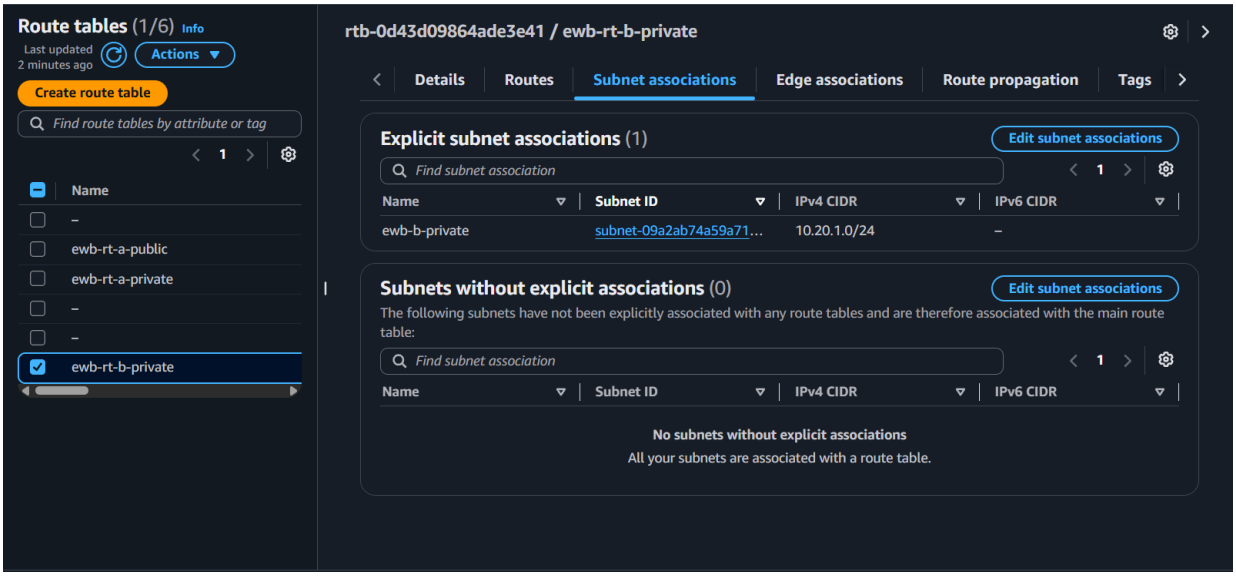



**Figure 10.** Console / ewb-rt-b-private > Subnet associations. Result: ewb-b-private 10.20.1.0/24 explicitly associated.

# 5. VPC peering connection

I created `ewb-a-b-peer` with VPC A as requester and VPC B as accepter (same account, same Region), then accepted the request. Acceptance is required even when both VPCs belong to the same account. Peering alone does not make ping or SSH work; the routes in Section 4 are also needed.

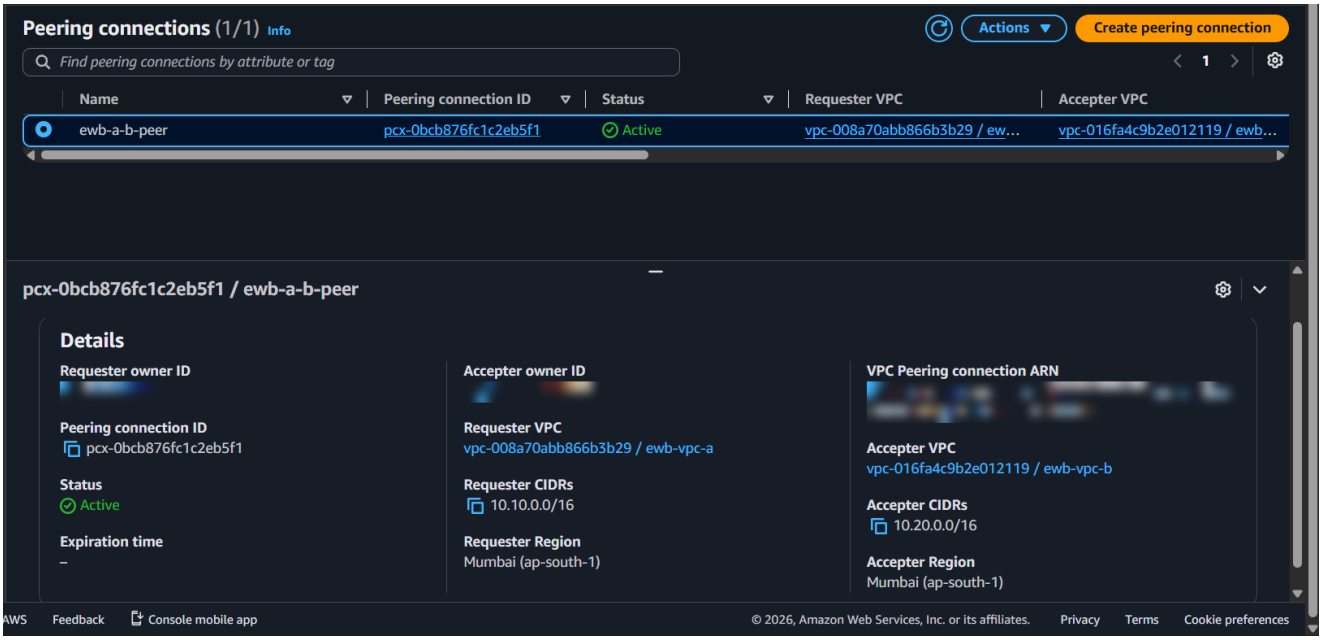



**Figure 11.** Console / VPC > Peering connections. ewb-a-b-peer is Active. Requester ewb-vpc-a 10.10.0.0/16, accepter ewb-vpc-b 10.20.0.0/16, both in Mumbai (ap-south-1). Account IDs and ARN blurred.

# 6. Security groups

I kept the default outbound allow-all rules. Security groups are stateful, so replies to permitted traffic are allowed automatically. I used /32 sources so only the individual lab hosts are allowed. SSH and ICMP are never open to 0.0.0.0/0.

| Security group | Type / port | Source |
|---|---|---|
| ewb-sg-bastion (VPC A) | SSH / TCP 22 | My current public IPv4 /32 (redacted) |
| ewb-sg-app-a (VPC A) | SSH / TCP 22 | 10.10.1.10/32 (bastion private IP) |
| ewb-sg-app-a (VPC A) | Custom ICMP IPv4, Echo request | 10.20.1.10/32 (App-B) |
| ewb-sg-app-b (VPC B) | SSH / TCP 22 | 10.10.1.10/32 (bastion private IP) |
| ewb-sg-app-b (VPC B) | Custom ICMP IPv4, Echo request | 10.10.2.10/32 (App-A) |

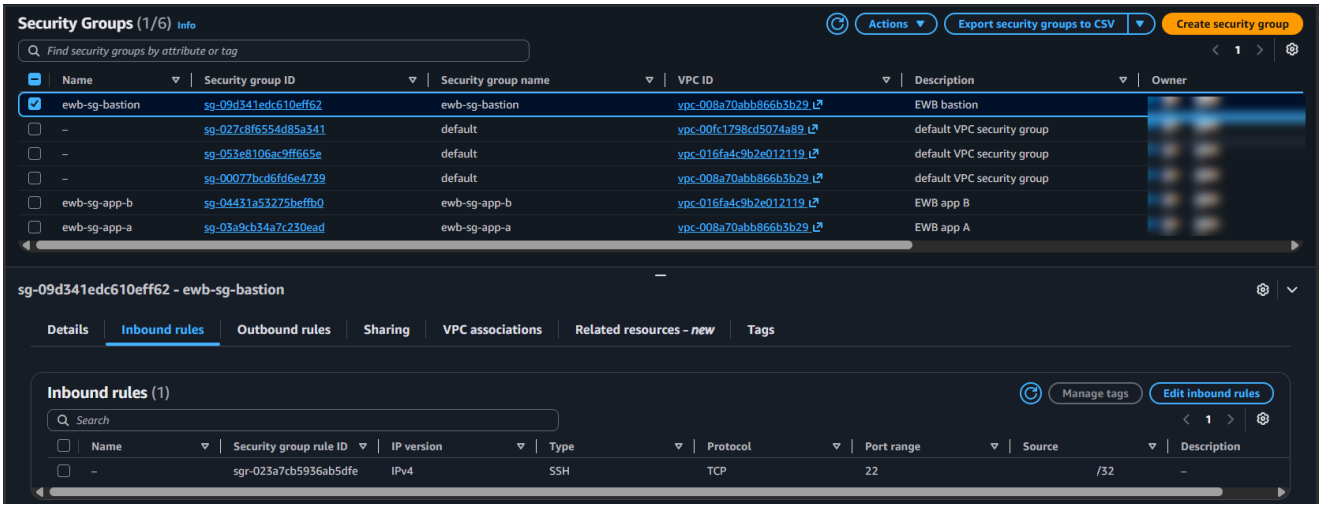



**Figure 12.** Console / ewb-sg-bastion > Inbound rules. Result: one rule, SSH TCP 22 from my /32 (IP and Owner column blurred).

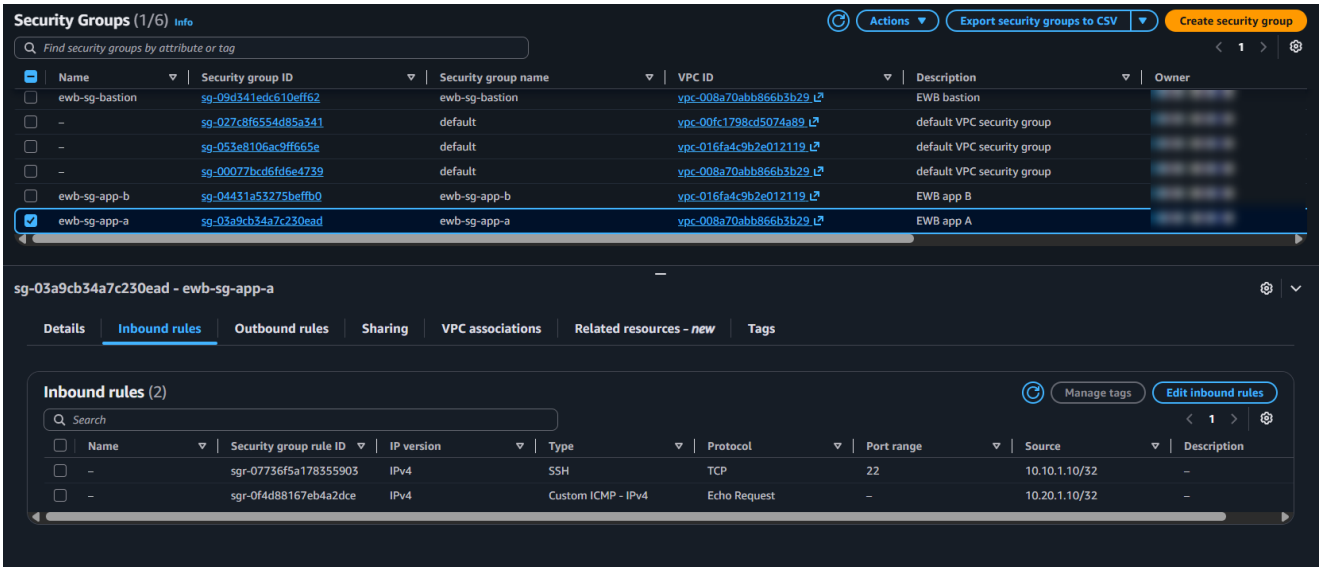



**Figure 13.** Console / ewb-sg-app-a > Inbound rules. Result: SSH from 10.10.1.10/32 and Echo Request from 10.20.1.10/32.

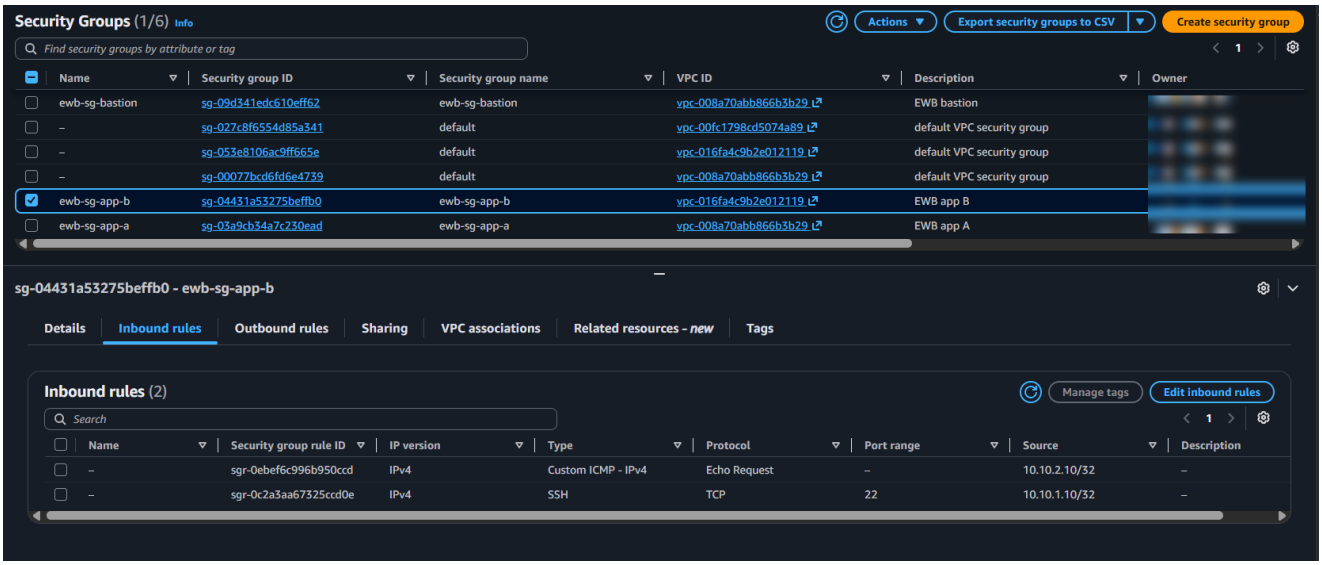



**Figure 14.** Console / ewb-sg-app-b > Inbound rules. Result: Echo Request from 10.10.2.10/32 and SSH from 10.10.1.10/32.

Port and source are different fields. SSH uses port 22. The private IP goes in Source, never in Port. App-B receives the bastion private IP as the SSH source when I connect through ProxyJump.

# 7. EC2 instances

Three Amazon Linux 2023 t3.micro instances share one key pair (`ewb-lab`) and the user ec2-user. The private key stays on my laptop and is never copied to any instance. Root volumes are 8 GB gp3 with delete-on-termination enabled. Only the bastion has a public IPv4 address.

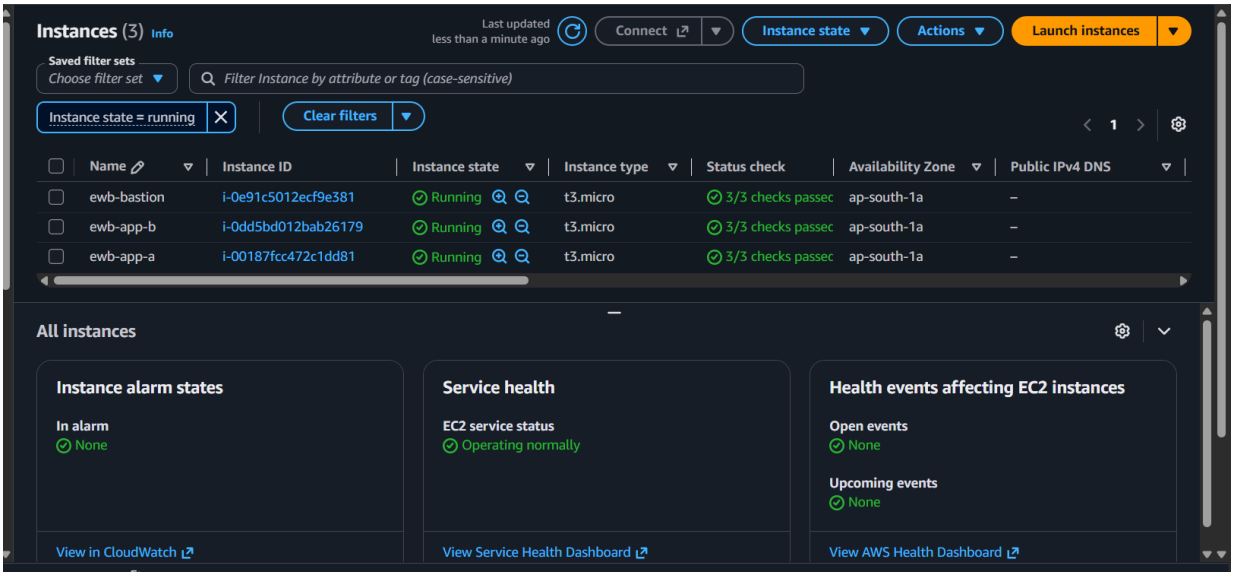



**Figure 15.** Console / EC2 > Instances. ewb-bastion, ewb-app-a and ewb-app-b are Running, t3.micro, 3/3 checks passed, all in ap-south-1a.

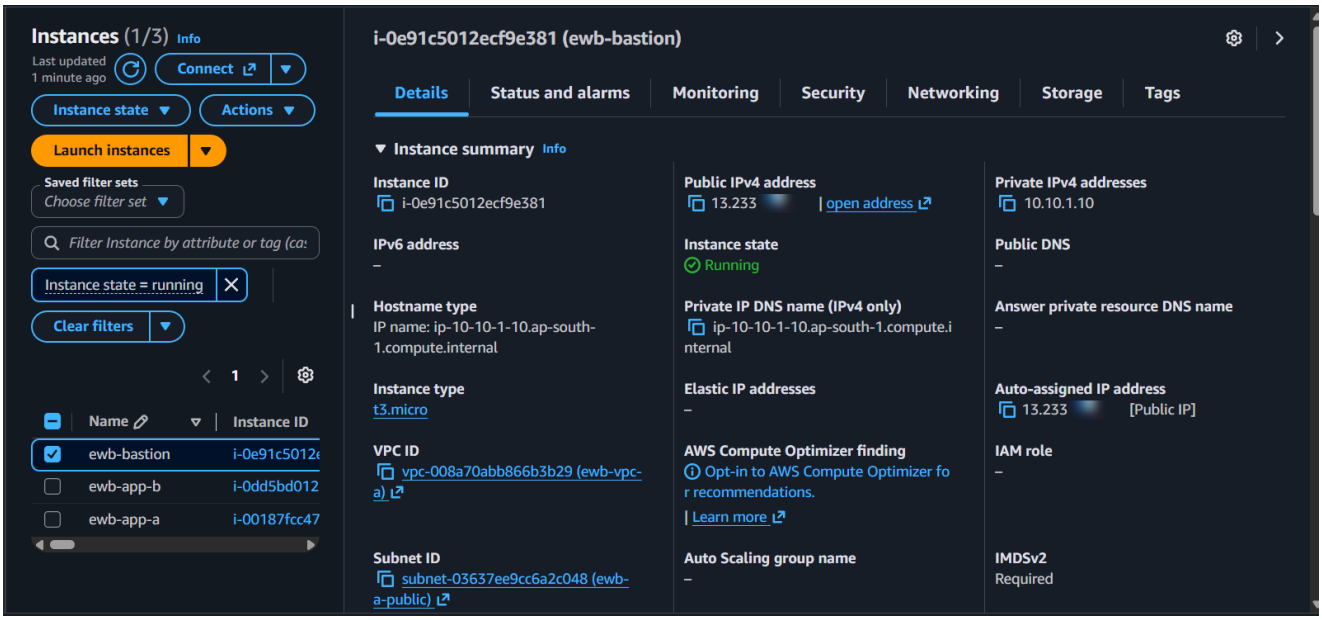



**Figure 16.** Console / ewb-bastion details. Private IPv4 10.10.1.10, public IPv4 assigned (last octets blurred), subnet ewb-a-public, IMDSv2 required.

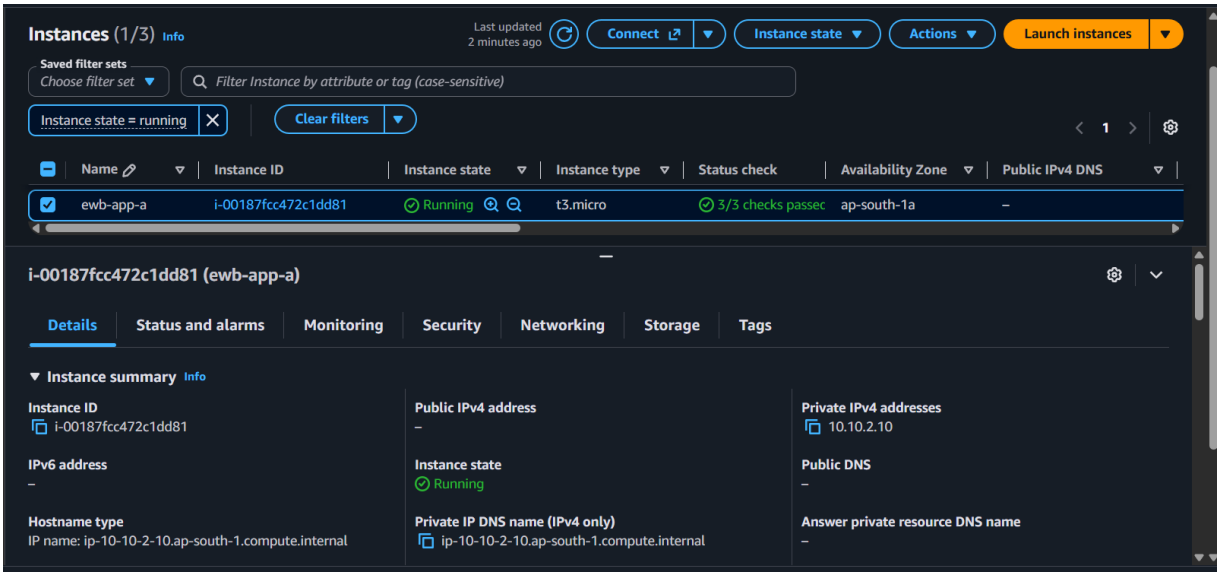



**Figure 17.** Console / ewb-app-a details. Private IPv4 10.10.2.10. Result: Public IPv4 address is empty (-).

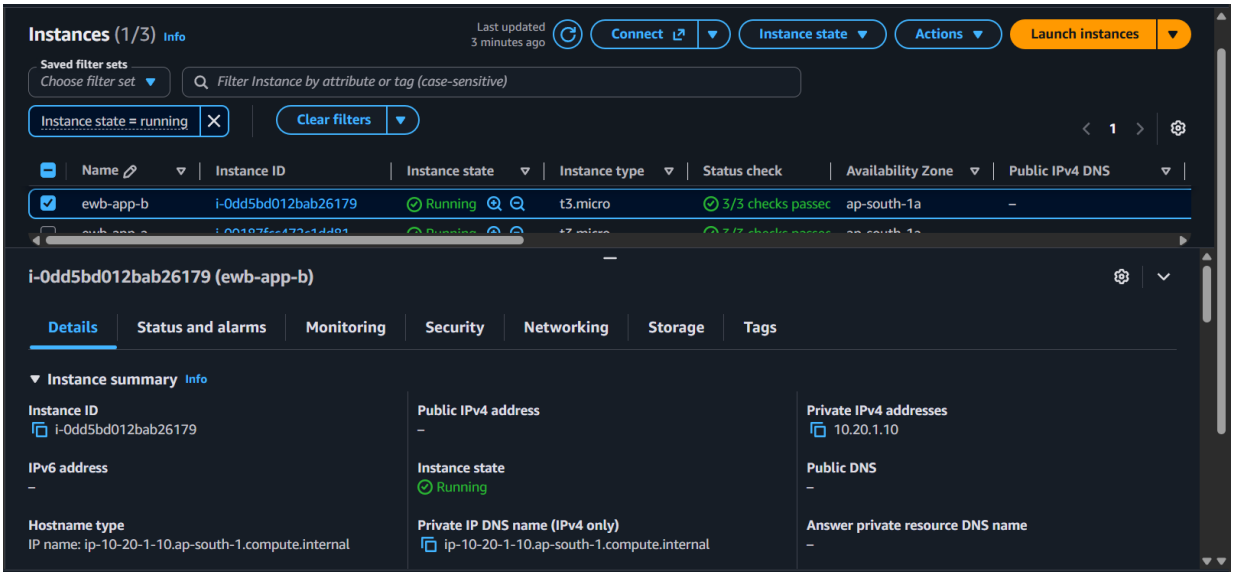



**Figure 18.** Console / ewb-app-b details. Private IPv4 10.20.1.10. Result: Public IPv4 address is empty (-).

# 8. Validation tests

All tests use private destination IPs. The expected results are the acceptance criteria; the right column is my own captured result.

| Test | Where / command | Expected | My result |
|---|---|---|---|
| 01 Baseline | App-A, before peering: ping -c 4 10.20.1.10 | No replies | 4 transmitted, 0 received, 100% packet loss |
| 02 Forward path | App-A, after routes: ping -c 4 10.20.1.10 | 4 replies; 0% loss | 4 received, 0% loss, avg rtt 0.292 ms |
| 03 Reverse path | App-B, after routes: ping -c 4 10.10.2.10 | 4 replies; 0% loss | 4 received, 0% loss, avg rtt 0.266 ms |
| 04 Private SSH | Laptop: ssh ewb-app-b | Shell on App-B through bastion | Connected; hostname ip-10-20-1-10.ap-south-1.compute.internal |
| 05 Isolation | App-A and App-B public IPv4 | No public IPv4 on either host | Public IP = None for both (console and CLI) |

## Test 01: baseline before peering

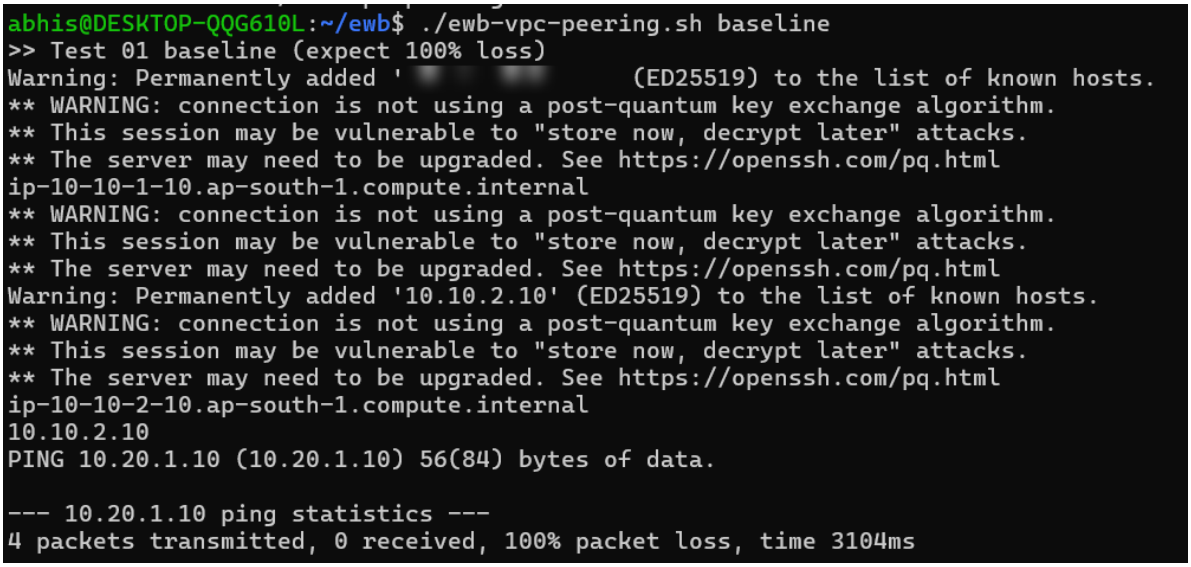



**Figure 19.** Host: laptop (WSL), ssh to ewb-bastion then ewb-app-a. Test 01 baseline. Result: bastion and App-A reachable through VPC A local route; ping to 10.20.1.10 shows 100% packet loss. Bastion public IP blurred.

## Tests 02 to 05: after peering and routes

```
abhis@DESKTOP-QQG610L:~/ewb$ ./ewb-vpc-peering.sh test
>> Test 02 forward
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
ip-10-10-2-10.ap-south-1.compute.internal
10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.578 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.198 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.199 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.194 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3141ms
rtt min/avg/max/mdev = 0.194/0.292/0.578/0.164 ms
>> Test 03 reverse
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Warning: Permanently added '10.20.1.10' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.180 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.205 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.470 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.210 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3128ms
rtt min/avg/max/mdev = 0.180/0.266/0.470/0.118 ms
>> Test 04 private SSH
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
ip-10-20-1-10.ap-south-1.compute.internal
10.20.1.10
>> Test 05 isolation (expect None)
-------------------------------------
|         DescribeInstances         |
+-----------+-------------+---------+
|  ewb-app-b |  10.20.1.10  |  None |
|  ewb-app-a |  10.10.2.10  |  None |
```

**Figure 20.** Host: App-A (forward), App-B (reverse), laptop (SSH), AWS CLI (isolation). Tests 02, 03, 04, 05. Result: both pings 4/4 with 0% loss, SSH shell on App-B (10.20.1.10), both apps show public IP None.

# 9. Controlled failure and recovery

To prove the security group is part of the path, I removed only the Echo request rule (ICMP from 10.10.2.10/32) from `ewb-sg-app-b`, kept the SSH rule, and repeated the forward ping from App-A. I then restored the exact same rule and repeated the test.

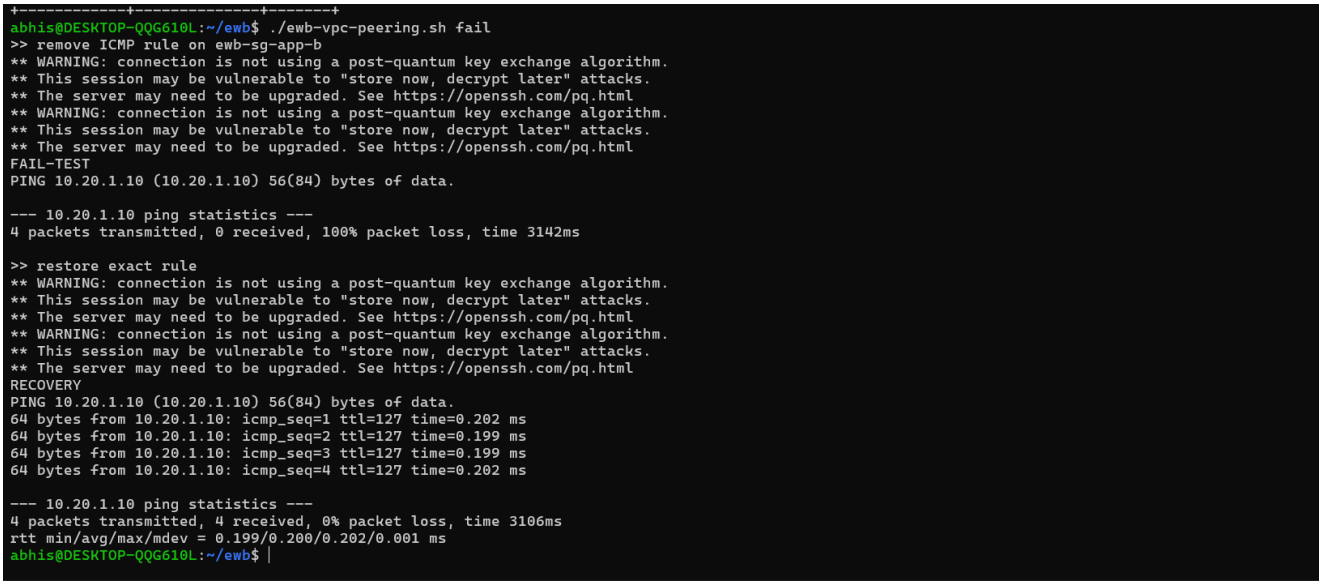



**Figure 21.** Host: App-A. Tests 06 and 07. Result: with the ICMP rule removed, 4 transmitted, 0 received, 100% loss. After restoring the exact rule, 4 received, 0% loss, avg rtt 0.200 ms.

**Explanation.** Routing was correct the whole time (peering Active, routes on both sides), yet the ping failed as soon as the destination security group stopped allowing ICMP Echo request from App-A. This shows that an Active peering connection does not by itself guarantee reachability: the route tables and the security group must all allow the traffic.

# 10. Forward path and return path

| Step | Forward: App-A to App-B (ping 10.20.1.10) | Return: App-B to App-A (reply) |
|---|---|---|
| 1 | App-A (10.10.2.10) sends the packet. ewb-rt-a-private matches 10.20.0.0/16 and selects the peering connection pcx-0bcb876fc1c2eb5f1. | App-B (10.20.1.10) sends the Echo reply to 10.10.2.10. ewb-rt-b-private matches 10.10.0.0/16 and selects the same peering connection. |
| 2 | The packet crosses the peering link into VPC B. | The reply crosses the peering link back into VPC A. |
| 3 | ewb-sg-app-b allows ICMP Echo request from 10.10.2.10/32, so the packet reaches App-B. | The security group is stateful, so the reply is allowed automatically. App-A receives it. |

**SSH to App-B through the bastion.** Laptop > internet gateway > bastion (ewb-sg-bastion allows TCP 22 from my /32). The bastion then opens a new SSH connection to 10.20.1.10: ewb-rt-a-public sends 10.20.0.0/16 to the peering connection, ewb-sg-app-b allows TCP 22 from 10.10.1.10/32, and ewb-rt-b-private returns the traffic to 10.10.0.0/16 over the same connection.

# 11. Viva questions

### 1. Why is the return route needed?

A peering connection is only the link. Each subnet route table must point the other VPC CIDR to it. If ewb-rt-b-private had no 10.10.0.0/16 route, App-B would only know its local 10.20.0.0/16 range, so its replies would have no path back to VPC A. Ping and SSH would fail even though the request arrived.

## 2. Why is the App-B SSH source the bastion private IP?

With ProxyJump the laptop does not connect to App-B directly. The bastion opens a separate SSH connection to App-B, so App-B sees the bastion private IP (10.10.1.10) as the source. Private IPs are preserved across peering, so ewb-sg-app-b allows TCP 22 only from 10.10.1.10/32 and not from my laptop public IP.

## 3. Why does Active peering not guarantee a successful ping?

Active only means the link exists. Traffic also needs routes on both sides, a security group rule that allows it (ICMP Echo request here), and network ACLs that allow it (default allow-all in this lab). My baseline test failed before routes existed, and my controlled failure failed after I removed the security group rule, both with peering Active.

## 4. Can VPC B use VPC A internet gateway over peering?

No. VPC peering is one-to-one and non-transitive. VPC B has no internet gateway, and traffic cannot use VPC A internet gateway, NAT or any other gateway through the peering connection. Peering only carries traffic between the two VPC CIDR ranges.

# 12. Cleanup

After saving all evidence I ran the cleanup step of my script. It terminates the three instances, deletes the lab key pair, deletes the peering connection, the lab security groups, the route table associations and route tables, the three subnets, detaches and deletes ewb-igw-a, and deletes both lab VPCs. It then lists any leftover resources with the project tag. The lab uses no NAT gateway and no Elastic IP.

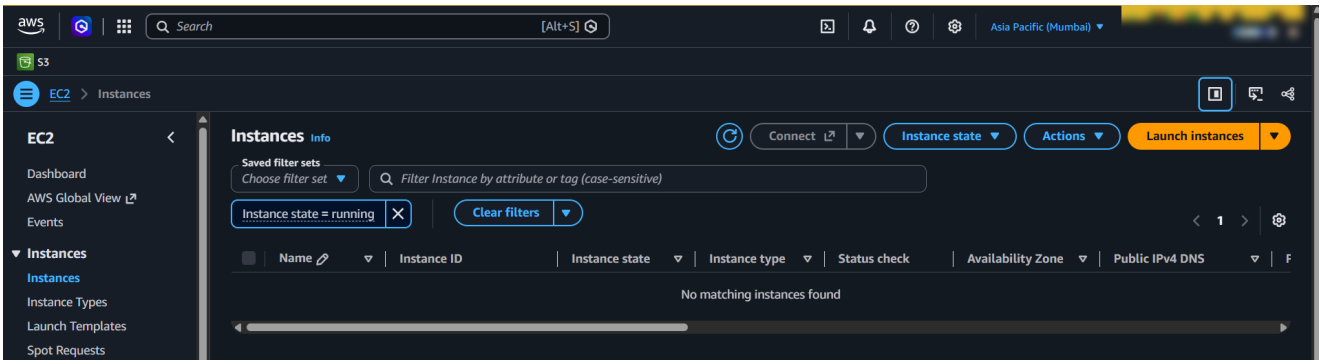



**Figure 22.** Console / EC2 > Instances (Mumbai), filter Instance state = running. Result: No matching instances found after cleanup. Account ID and username blurred.

# 13. What I learned

Peering is a link, not a route: both sides need routes and the right security group rules. A failed test before peering is useful evidence, because it proves the later success comes from the peering and routes and not from something else. Using /32 sources and no public IP on the app servers keeps the design tight, and a bastion with ProxyJump means the private key never leaves my laptop. Scripting the build with the AWS CLI made the lab repeatable and made cleanup safer, because every resource was created with the same tags.